

Rapport technique - POC RAG Puls-Events

Projet 11 OpenClassrooms - Assistant de recommandation d'événements culturels

Périmètre : OpenAgenda Culture Versailles - UID 60871835

1. Contexte et objectifs

Puls-Events souhaite tester l'intégration d'un assistant conversationnel capable de recommander des événements culturels à partir de données publiques. Le POC développé permet à un utilisateur de formuler une question en langage naturel, par exemple : « Je cherche un concert à Versailles », et d'obtenir une réponse fondée sur des événements réellement collectés.

- Collecter des événements depuis OpenAgenda.
- Nettoyer et structurer les données collectées.
- Transformer les événements en documents exploitables par LangChain.
- Générer des embeddings avec Mistral et créer un index FAISS.
- Fournir une interface Streamlit pour la démonstration.
- Ajouter des tests unitaires et une évaluation automatique simplifiée du RAG.

2. Architecture générale

L'architecture suit le principe Retrieval-Augmented Generation : le LLM ne répond pas uniquement à partir de ses connaissances internes. La question utilisateur est d'abord utilisée pour récupérer des événements pertinents dans une base vectorielle FAISS ; ces événements sont ensuite injectés dans le prompt envoyé à Mistral.

Composant	Rôle
OpenAgenda API	Récupération des événements Culture Versailles.
Pré-processing	Filtrage sur Versailles, date de moins d'un an ou à venir, nettoyage et suppression des doublons.
Documents LangChain	Un événement devient un Document avec page_content et metadata.
Mistral Embed	Transformation du texte en vecteur numérique.
FAISS	Indexation locale et recherche par similarité sémantique.
Mistral Small	Génération de la réponse contextualisée.
Streamlit	Interface de démonstration du chatbot.

3. Collecte et nettoyage des données

La source de données est l'agenda public OpenAgenda « Culture Versailles ». Le script `src/extract_openagenda.py` interroge l'API, applique les règles de qualité, puis sauvegarde le dataset nettoyé dans `data/events-versailles.csv`.

Règle	Objectif
Ville = Versailles	Garantir le périmètre géographique retenu.
Date récente ou future	Respecter la contrainte des événements de moins d'un an.
Titre et description non vides	Garantir un contenu exploitable par le RAG.
Doublons supprimés	Éviter de retrouver plusieurs fois le même événement.

Le dataset final contient 98 événements et les colonnes title, description, city, date, begin_date et url. La colonne begin_date est au format ISO 8601, ce qui permet à Pandas de vérifier automatiquement la contrainte temporelle.

4. Chunking et documents LangChain

Les données ne sont pas des PDF ni de longs rapports, mais des fiches événementielles courtes. La stratégie retenue est donc un chunking par unité métier : un événement OpenAgenda correspond à un Document LangChain et donc à un chunk unique.

1 événement OpenAgenda -> 1 Document LangChain -> 1 embedding -> FAISS

- page_content : texte complet utilisé pour générer l'embedding.
- metadata : informations structurées conservées pour l'affichage et un éventuel filtrage ultérieur.

5. Vectorisation et index FAISS

Le script src/build_faiss.py lit le CSV nettoyé, crée les Documents LangChain, génère les embeddings avec le modèle mistral-embed et sauvegarde l'index FAISS dans vectorstore/faiss_versailles.

Fichier	Rôle
index.faiss	Contient l'index vectoriel FAISS.
index.pkl	Conserve les Documents LangChain et leurs métadonnées.

6. Système RAG et interface

L'application app.py charge l'index FAISS, récupère les quatre documents les plus proches de la question utilisateur, construit un contexte et demande à Mistral de générer une réponse uniquement à partir de ce contexte. L'interface Streamlit permet une démonstration live simple.

- Question utilisateur en langage naturel.
- Recherche FAISS avec similarity_search(k=4).
- Construction du contexte avec les événements retrouvés.
- Génération de la réponse avec mistral-small-latest.

7. Tests unitaires et pipeline

Les tests Pytest sont intégrés dans tests/test_events.py et peuvent être lancés directement avec python -m pytest ou via scripts/run_tests.py. Ils ne font aucun appel réseau.

- Existence du dataset nettoyé et dataset non vide.
- Présence des colonnes obligatoires.
- Tous les événements sont localisés à Versailles.
- Tous les événements sont récents de moins d'un an ou à venir.
- Absence de champs obligatoires vides et absence de doublons.
- Existence du jeu de test annoté.
- Présence des fichiers index.faiss et index.pkl.

8. Évaluation automatique du RAG

Le fichier data/evaluation_dataset.csv contient des couples question/réponse attendue. Le script evaluate_rag.py exécute le RAG sur ces questions et produit data/evaluation_results.csv.

Métrique	Interprétation
semantic_similarity	Proximité sémantique entre la réponse générée et la réponse attendue.
faithfulness	Fidélité de la réponse au contexte récupéré dans FAISS.

Les derniers résultats obtenus sont une similarité sémantique moyenne de 0,829 et une fidélité moyenne au contexte de 0,917. Ces scores indiquent que les réponses sont globalement proches des attentes et fortement ancrées dans les documents récupérés.

9. Limites et recommandations

- Le périmètre est limité à Versailles et à un seul agenda OpenAgenda.
- La qualité dépend des descriptions fournies par OpenAgenda.
- L'index est reconstruit manuellement dans le POC.
- Le système ne gère pas encore de mémoire conversationnelle ni de re-ranking.
- Une version finale pourrait automatiser la mise à jour des données et améliorer l'évaluation avec des réponses de référence plus précises.

10. Conclusion

Le POC démontre la faisabilité d'un assistant RAG pour recommander des événements culturels. Le projet est reproductible, documenté, testé et doté d'une interface de démonstration. Il constitue une base solide pour une industrialisation future chez Puls-Events.